1 Architecture

So, you might be wondering what Libra is, Libra is a kernel put simply, its goal is to be simple and efficient. Libra also targets abstraction. Libra uses the Limine bootloader as its primary, which gives the kernel perks, including:

- Paging setup, For more information, see Section 1.5.1 [Paging], page 1.
- Framebuffer setup
- Modules

Libra also happens to use the embedded libc, newlib. Although it is not stock, we have fit the libc with bunches more POSIX and UNIX implementations and headers, many of the syscalls have needed shims in the userspace implementation to meet POSIX requirements. We recommend using the Libra SDK to compile your programs.

1.1 HVFS (Hierarchical Variable and Function System)

HVFS is the main registry of Libra/Carrera which has variables, a VFS like structure, including keys that can contain other keys it supports uint64_t and a bunch more types, aswell it supports syscalls. Which will hopefully be added in a version sometime.

1.2 Why You Should Use Libra

Libra is incredibly efficient, and is improving day-by-day, getting constant security updates. Libra is also incredibly lightweight. Libra is efficient partly due to its low memory usage and lightweightness. Libra also includes many optimizations, including hardware rendering.

1.3 Why You Should Develop for Libra

Developing for Libra is incredibly easy due to our abstractions. We have decided to make the interface as simple as possible without any cluttering, not adding useless syscalls. We also have a dedicated guide for developers, See: Chapter 4 [Guide For Developers], page 7,

1.4 Features

Libra has countless features, including USB support, NVMe support, HDD support, RTL8139 support. On the software side, our syscall reference is very broad, giving you many possibilities on Libra.

1.5 Virtual Memory Manager and Memory Management

1.5.1 Paging

We have chosen higher-level paging, which makes the kernel safer for use and simpler. Unlike identity mapping which lets all processes access any part of memory, higher-level paging

has explicit permission settings. Each process gets its own CR3 (address space) meaning that for example, 0x60000 in Process A has different contents (e.g. 69). While Process B has different contents than Process A (e.g. 67).

1.5.2 Allocator

The VMM Allocator uses a bitmap at a specific address, see `src/alloc/alloc.c` for more information.

2 Syscall Table

Name	Index	Parameters & Return	Description
sys_read	0	fd, buf, count, offset -> size/err	Reads bytes from file descriptor into user buffer.
sys_write	1	fd, buf, count -> size/err	Writes bytes from buffer to file descriptor/TTY.
sys_open	2	path, flags, mode -> fd/err	Opens VFS file or device node.
sys_mkdir	3	path, mode -> status/err	Creates a new VFS directory.
sys_rmdir	4	path -> status/err	Removes an empty VFS directory.
sys_close	5	fd -> status/err	Closes an open file descriptor.
sys_move_file	6	fd, new_path -> status/err	Relocates/renames an open file descriptor.
sys_create_file	7	data, path, flags -> fd/err	Creates a regular file or device node.
sys_delete_file	8	path -> status/err	Removes a file from VFS.
sys_get_perm_key	9	target_level, perm_out -> status	Signs and retrieves process permission key.
sys_graduate	10	none -> status	ABI filler.
sys_draw_rect	11	x, y, w, h, r, g, b -> status	Draws colored rectangle on primary framebuffer.
sys_exit_handler	12	none -> status	Kernel handler for process exit state.
sys_ipc_recv	13	buf, size, sender_pid -> status	Blocking IPC message reception.
sys_ipc_send	14	target_pid, buf, size -> status	Blocking IPC message transmission.
sys_getpid	15	none -> pid	Returns current process ID.
sys_terminate	16	pid -> status/err	Terminates process by PID.
sys_fstat	17	fd, st -> status/err	Retrieves file metadata statistics.
sys_read_stdin	18	buf, count -> bytes_read	Reads keyboard scancodes from stdin.
sys_spawn	19	path, flags, argv, envp -> pid/err	Spawns new process image from ELF path.
sys_waitpid	20	pid -> status/err	Waits for process state change.
sys_draw_image	21	x, y, w, h, img_buf -> status	Renders raw image buffer to framebuffer.
sys_mmap	22	addr, len, prot, flags, fd, offset -> virt_addr	Maps virtual memory region.
sys_munmap	23	addr, len -> status/err	Unmaps virtual memory pages.
sys_set_signal_handler	24	sig, handler -> status/err	Registers process signal handler callback.
sys_send_signal	25	pid, sig -> status/err	Sends signal to target process.
sys_read_mouse	26	buf, count -> bytes_read	Reads raw mouse input events.

sys_get_pixel	27	x, y, r_out, g_out, b_out -> status	Queries pixel RGB values at frame coordinate.
sys_ipc_send_nonblock	28	target_pid, buf, size -> status	Non-blocking IPC message transmission.
sys_ipc_recv_nonblock	29	buf, size, sender_pid -> status	Non-blocking IPC message reception.
sys_socket	30	domain, type -> sock_idx/err	Allocates a network socket.
sys_socket_connect	31	sock_idx, addr -> status/err	Connects network socket to remote address.
sys_socket_recv	32	sock_idx, buf, len -> bytes_read	Receives data from socket.
sys_socket_send	33	sock_idx, buf, len -> bytes_sent	Transmits data via socket.
sys_socket_close	34	sock_idx -> status/err	Closes network socket.
sys_get_ticks	35	none -> ticks	Returns system timer tick count (*10).
sys_sleep_ms	36	ms -> status	Delays thread execution.
sys_get_launchd_pid	37	none -> pid	Returns PID of system init/launchd daemon.
sys_uname	38	utsname_ptr -> status/err	Populates system identification structures.
sys_sethostname	39	name, len -> status/err	Updates kernel system hostname.
sys_gethostname	40	dest, len -> status/err	Retrieves system hostname.
sys_rtc_get_time	41	timespec_ptr -> status	Fetches current hardware real-time clock.
sys_listdir	42	path, out_buf, max -> count/err	Enumerates VFS directory contents.
sys_reboot	43	none -> status	Triggers system hardware reboot via ACPI.
sys_poweroff	44	none -> status	Enters ACPI S5 system poweroff state.
sys_delete_file_alias	45	path -> status/err	VFS file deletion alias.
sys_ioctl	46	fd, request, argp -> status/err	Performs device control operation.
sys_hvfs_create	47	path -> status/err	Creates HVFS variable/function node.
sys_hvfs_set_type	48	path, type -> status/err	Sets node type in HVFS tree.
sys_hvfs_set	49	path, data, size -> status/err	Writes raw data to HVFS node.
sys_hvfs_get	50	path, buf, size -> status/err	Reads raw data from HVFS node.
sys_hvfs_get_type	51	path, type_out -> status/err	Queries data type of HVFS node.
sys_hvfs_remove	52	path -> status/err	Removes node from HVFS tree.

sys_hvfs_listdir	53	path, buf, size -> status/err	Enumerates HVFS directory branch.
sys_hvfs_stat	54	path, stat_out -> status/err	Retrieves HVFS node metadata.
sys_chdir	55	path -> status/err	Changes current process working directory.
sys_getcwd	56	buf, size -> status/err	Gets current process working directory.
sys_realpath	57	path, resolved_path -> path_ptr	Resolves canonicalized VFS path.
sys_ps	58	utask_buf, max -> count/err	Fetches active process table entries.
sys_tty_clear	59	none -> status	Clears current TTY display buffer.
sys_tty_switch	60	tty_idx -> status	Switches active virtual TTY screen.
sys_tty_draw_pixel	61	x, y, color -> status	Plots single pixel on TTY screen.
sys_tty_draw_img	62	x, y, buf, w, h -> status	Draws pixel array to TTY frame.
sys_tty_draw_rect	63	x, y, w, h, color -> status	Draws filled rectangle on TTY frame.
sys_get_tls	64	slot_key -> val	Reads thread-local storage key value.
sys_set_tls	65	slot_key, val -> status	Sets thread-local storage key value.
sys_clone	66	entry, stack, arg -> thread_id	Spawns kernel/user thread context.
sys_join	67	thread_id, status_out -> status	Blocks until thread execution finishes.
sys_pci_lookup	68	buf, max_count -> count	Enumerates detected PCI hardware devices.
sys_thread_exit	69	exit_code -> none	Terminates current executing thread.
sys_fork	70	none -> child_pid/0	Duplicates current process image.
sys_execve	71	path, argv, envp -> status/err	Replaces process image with ELF binary.
sys_link	72	oldpath, newpath -> status/err	Creates hard link to VFS target.
sys_socket_listen	73	sock_idx, addr -> status/err	Marks network socket as passive listener.
sys_socket_accept	74	sock_idx -> new_sock_idx/err	Accepts incoming socket connection.
sys_socket_bind	75	sock_idx, addr -> status/err	Binds network socket to local address.

3 Boot Chain

This chapter is explaining the chain of boot in chronological order.

1. The kernel sets up the GDT, which is already set up by Limine, but we change it so our segment selectors are standard and so we can also use Ring 3 (aka. userspace) which makes it so processes cannot do high privileged tasks like ("hlt") or ("cli"), otherwise the GP fault is triggered, see [IDT], page 6.
2. The kernel sets up the IDT, which catches all of the faults and prevents instant reboot on things like #DE (Divide By Zero Error)
3. The kernel then gets the memory usable and reclaimable regions for later use
4. The kernel then initializes the VMM & PMM
5. The kernel then initializes the keyboard although its interrupts will be **ignored** until APIC initialization
6. The kernel initializes the TTY system and switches to TTY1
7. The kernel initializes the VFS
8. uACPI is initialized
9. APIC setup is started
10. SSE is enabled
11. The scheduler is initialized
12. The PCIe bus is scanned.
13. Multiple drivers like NVMe are initialized
14. The tests are started
15. The main kernel task is started
16. The scheduler is started
17. launchd is started as PID 2
18. All the services in the config are launched

4 Guide For Developers

4.1 Getting Started...

4.1.1 Cloning the Repo

If you're reading this on the website, then go ahead and clone `https://www.github.com/Sysdev32/libra.git` If you have already cloned this skip to Section 4.1.2 [Getting the SDK], page 7,

4.1.2 Getting The SDK

The SDK contains the libc and lots of libraries for you to link your program against. To obtain the SDK go to the releases and select your release, then download the sdk.tar.xz listed in the assets.

4.2 Compiling your program

Since we do not have a dedicated portable toolchain yet, the developer has to use the standard generic x86_64-elf cross-compilation toolchain with these flags:

- -I<the sysroot include>
- -nostdlib
- -L<the sysroot libraries>
- -lc

Append whichever flags you need after this.